

Rust

Perfectionnement

Haute Performance

Rust Haute Performance

Systemes Distribués

Agenda du Jour 3

Horaire	Module	Durée
09:00 – 11:00	Module 3.1 – Sérialisation haute performance	2h
11:00 – 13:00	Module 3.2 – gRPC avec Tonic	2h
14:00 – 16:00	Module 3.3 – Patterns distribués	2h
16:00 – 17:00	Module 3.4 – Résilience	1h

“ **Objectif** : Concevoir des services distribués performants et résilients en Rust ”

Module 3.1 – Sérialisation Haute Performance

Comparatif des formats

Format	Taille	Encode	Decode	Zero-copy	Schema
JSON	Grand	Lent	Lent	Non	Non
Bincode	Petit	Rapide	Rapide	Non	Non
MessagePack	Moyen	Rapide	Rapide	Non	Non
Protobuf	Petit	Rapide	Rapide	Non	Oui
FlatBuffers	Moyen	Rapide	Instant	Oui	Oui
Cap'n Proto	Moyen	Rapide	Instant	Oui	Oui

“ Choisir selon le ratio fréquence d'accès / taille des messages ”

Protocol Buffers avec prost

```
// proto/metrics.proto
syntax = "proto3";
package ovh.metrics;

message MetricReport {
    string hostname = 1;
    int64 timestamp = 2;
    repeated Metric metrics = 3;
}

message Metric {
    string name = 1;
    double value = 2;
    map<string, string> labels = 3;
}
```

Protocol Buffers avec prost

```
// build.rs
fn main() {
    prost_build::compile_protos(&["proto/metrics.proto"], &["proto/"]).unwrap();
}

// src/main.rs
mod proto { include!(concat!(env!("OUT_DIR"), "/ovh.metrics.rs")); }

use proto::{MetricReport, Metric};

let report = MetricReport {
    hostname: "node-01".into(),
    timestamp: chrono::Utc::now().timestamp(),
    metrics: vec![Metric {
        name: "cpu_usage".into(),
        value: 65.3,
        labels: [("core".into(), "0".into())].into(),
    }],
};

// Encode (Vec<u8>)
let bytes = report.encode_to_vec();
// Decode
let decoded = MetricReport::decode(bytes.as_slice())?;
```

FlatBuffers : Zero-copy access

```
// Pas de désérialisation : accès direct dans le buffer
use flatbuffers::{FlatBufferBuilder, WIPOffset};

let mut builder = FlatBufferBuilder::new();
let hostname = builder.create_string("node-01");

let metric = MetricFb::create(&mut builder, &MetricFbArgs {
    name: Some(builder.create_string("cpu_usage")),
    value: 65.3,
});

// Lecture : accès direct au buffer, 0 copie
let report = flatbuffers::root::<MetricReportFb>(buffer)?;
println!("Host: {}", report.hostname()); // Pointe dans le buffer original
```

Quand utiliser FlatBuffers : Données volumineuses où on n'accède qu'à quelques champs (logs, events)

Module 3.2 – gRPC avec Tonic

Définition de service

```
// proto/kvstore.proto
syntax = "proto3";
package ovh.kvstore;

service KvStore {
  rpc Get(GetRequest) returns (GetResponse);
  rpc Put(PutRequest) returns (PutResponse);
  rpc Delete(DeleteRequest) returns (DeleteResponse);
  rpc Watch(WatchRequest) returns (stream WatchEvent); // Server streaming
  rpc PutStream(stream PutRequest) returns (PutResponse); // Client streaming
}

message GetRequest { string key = 1; }
message GetResponse {
  bytes value = 1;
  bool found = 2;
}
message PutRequest {
  string key = 1;
  bytes value = 2;
}
message PutResponse { bool success = 1; }
```

Implémentation serveur Tonic

```
use tonic::{Request, Response, Status};
use proto::kv_store_server::{KvStore, KvStoreServer};

pub struct KvStoreService {
    store: Arc<DashMap<String, Vec<u8>>>,
}

#[tonic::async_trait]
impl KvStore for KvStoreService {
    async fn get(&self, request: Request<GetRequest>)
        -> Result<Response<GetResponse>, Status>
    {
        let key = &request.get_ref().key;
        match self.store.get(key) {
            Some(entry) => Ok(Response::new(GetResponse {
                value: entry.value().clone(),
                found: true,
            })),
            None => Ok(Response::new(GetResponse {
                value: vec![],
                found: false,
            })),
        }
    }

    type WatchStream = ReceiverStream<Result<WatchEvent, Status>>;

    async fn watch(&self, request: Request<WatchRequest>)
        -> Result<Response<Self::WatchStream>, Status>
    {
        let (tx, rx) = mpsc::channel(128);
        // Spawner un watcher qui envoie des events
        tokio::spawn(async move { /* ... */ });
        Ok(Response::new(ReceiverStream::new(rx)))
    }
}

// Lancer le serveur
Server::builder()
    .add_service(KvStoreServer::new(KvStoreService::new()))
    .serve("0.0.0.0:50051".parse())
```

Streaming bidirectionnel

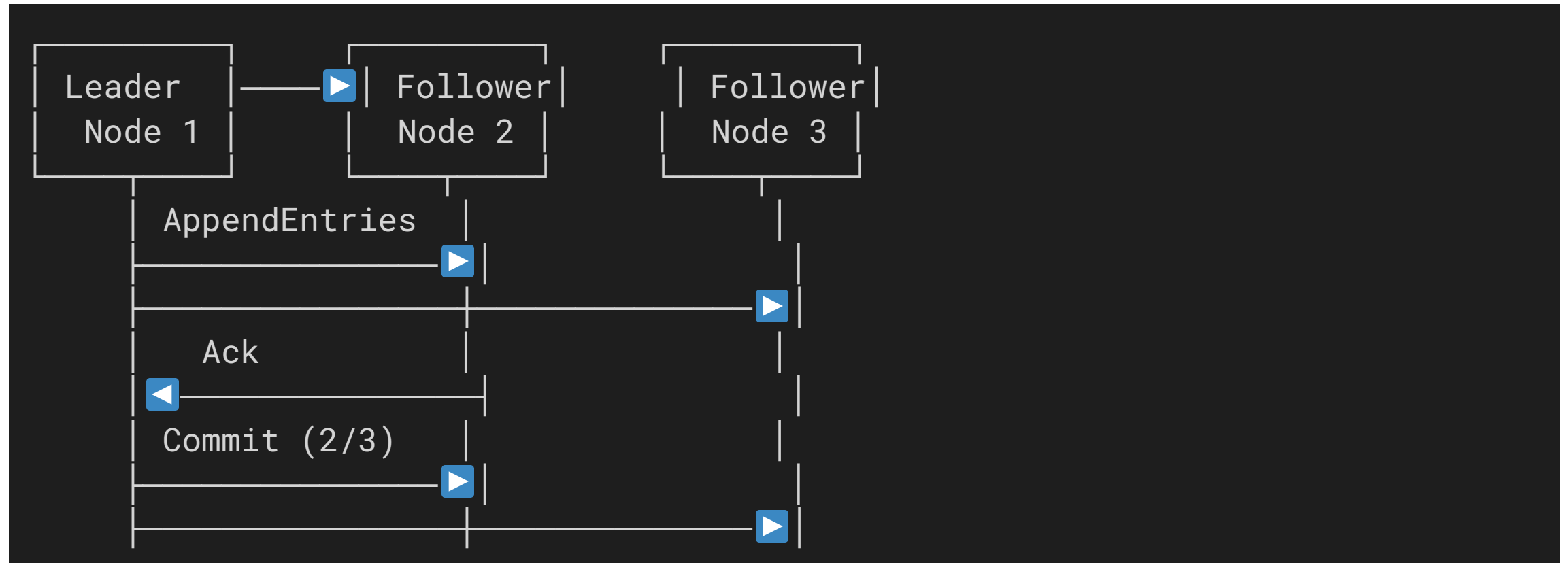
```
// Client streaming + Server streaming = Bidirectionnel
async fn sync_data(
    &self,
    request: Request<tonic::Streaming<SyncRequest>>,
) -> Result<Response<Self::SyncDataStream>, Status> {
    let mut in_stream = request.into_inner();
    let (tx, rx) = mpsc::channel(128);

    tokio::spawn(async move {
        while let Some(Ok(req)) = in_stream.next().await {
            // Traiter chaque requête
            let response = process_sync(req).await;
            if tx.send(Ok(response)).await.is_err() {
                break; // Client déconnecté
            }
        }
    });

    Ok(Response::new(ReceiverStream::new(rx)))
}
```

Module 3.3 – Patterns Distribués

Consensus Raft (openraft)



- **Leader election** : Un seul leader à la fois (terms)
- **Log replication** : Le leader réplique les entrées
- **Commit** : Quorum (majorité) pour confirmer

Consistent Hashing pour sharding

```
use std::collections::BTreeMap;
use std::hash::{Hash, Hasher};

struct ConsistentHash {
    ring: BTreeMap<u64, String>, // position → node_id
    virtual_nodes: usize,
}

impl ConsistentHash {
    fn new(virtual_nodes: usize) -> Self {
        Self { ring: BTreeMap::new(), virtual_nodes }
    }

    fn add_node(&mut self, node: &str) {
        for i in 0..self.virtual_nodes {
            let key = format!("{}", i, node);
            let hash = self.hash(&key);
            self.ring.insert(hash, node.to_string());
        }
    }

    fn get_node(&self, key: &str) -> Option<&str> {
        if self.ring.is_empty() { return None; }
        let hash = self.hash(key);
        // Trouver le premier noeud >= hash (ou le premier du ring)
        let node = self.ring.range(hash..)
            .next()
            .or_else(|| self.ring.iter().next())
            .map(|(_, v)| v.as_str());
        node
    }

    fn hash(&self, key: &str) -> u64 {
        let mut hasher = std::collections::hash_map::DefaultHasher::new();
        key.hash(&mut hasher);
        hasher.finish()
    }
}
```

CRDTs : Convergence sans coordination

```
/// G-Counter : Grow-only counter (CRDT)
/// Chaque noeud a son propre compteur, le total = somme de tous
#[derive(Clone, Debug)]
struct GCounter {
    node_id: String,
    counters: HashMap<String, u64>,
}

impl GCounter {
    fn increment(&mut self) {
        *self.counters.entry(self.node_id.clone()).or_insert(0) += 1;
    }

    fn value(&self) -> u64 {
        self.counters.values().sum()
    }

    /// Merge : prendre le max de chaque compteur (commutatif, associatif, idempotent)
    fn merge(&mut self, other: &GCounter) {
        for (node, &count) in &other.counters {
            let entry = self.counters.entry(node.clone()).or_insert(0);
            *entry = (*entry).max(count);
        }
    }
}
```


Module 3.4 – Résilience

Circuit Breaker

```
use std::sync::atomic::{AtomicU32, AtomicU64, Ordering};
use std::time::{Duration, Instant};

#[derive(Debug, Clone, Copy, PartialEq)]
enum CircuitState { Closed, Open, HalfOpen }

struct CircuitBreaker {
    failure_count: AtomicU32,
    failure_threshold: u32,
    reset_timeout: Duration,
    last_failure: Mutex<Option<Instant>>,
}

impl CircuitBreaker {
    async fn call<F, T, E>(&self, f: F) -> Result<T, CircuitBreakerError<E>>
    where
        F: Future<Output = Result<T, E>>,
    {
        match self.state() {
            CircuitState::Open => Err(CircuitBreakerError::Open),
            CircuitState::HalfOpen | CircuitState::Closed => {
                match f.await {
                    Ok(result) => {
                        self.on_success();
                        Ok(result)
                    }
                    Err(e) => {
                        self.on_failure();
                        Err(CircuitBreakerError::Inner(e))
                    }
                }
            }
        }
    }
}
```

Retry avec backoff exponentiel

```
use std::time::Duration;
use tokio::time::sleep;

pub struct RetryPolicy {
    pub max_retries: u32,
    pub initial_delay: Duration,
    pub max_delay: Duration,
    pub multiplier: f64,
}

pub async fn retry_with_backoff<F, Fut, T, E>(
    policy: &RetryPolicy,
    mut f: F,
) -> Result<T, E>
where
    F: FnMut() -> Fut,
    Fut: Future<Output = Result<T, E>>,
    E: std::fmt::Display,
{
    let mut delay = policy.initial_delay;

    for attempt in 0..=policy.max_retries {
        match f().await {
            Ok(result) => return Ok(result),
            Err(e) if attempt < policy.max_retries => {
                tracing::warn!(
                    attempt = attempt + 1,
                    max = policy.max_retries,
                    delay_ms = delay.as_millis(),
                    error = %e,
                    "Retry after failure"
                );
                sleep(delay).await;
                delay = Duration::from_secs_f64(
                    (delay.as_secs_f64() * policy.multiplier).min(policy.max_delay.as_secs_f64())
                );
                // Jitter
                let jitter = rand::random:::<f64>() * 0.3;
                delay = Duration::from_secs_f64(delay.as_secs_f64() * (1.0 + jitter));
            }
            Err(e) => return Err(e),
        }
    }
    unreachable!()
}
```

Récapitulatif Jour 3

Points clés

- Sérialisation : protobuf (prost), FlatBuffers (zero-copy)
- gRPC : Tonic, streaming, interceptors
- Consensus Raft, consistent hashing, CRDTs
- Résilience : circuit breaker, retry backoff

Demain : Projet Fil Rouge – KV Store Distribué

